

# Cambios Fase 2 - AhorraGo

Fecha: 2026-05-31

## Objetivo

Reducir deuda técnica sin romper el MVP, enfocándose en `main.py`, normalización duplicada, matching de productos, endpoints pesados y tests de integración.

## Cambios Aplicados

### 1. Normalización compartida

- Se creó `app/normalizacion.py`.
- Se movieron funciones de normalización y comparación de claves desde `main.py`.
- `importar_csv.py` ahora usa `generar_producto_base` desde el módulo compartido.
- Se mantuvo compatibilidad con imports existentes usados por tests.
- Se normalizaron equivalencias de formato:
  - 1L y 1000ml.
  - 500g y 0.5kg.
  - pack 6 y 6 unidades.

Archivos:

- `app/normalizacion.py`
- `app/main.py`
- `app/importar_csv.py`

### 2. Matching de productos

- Se creó `app/matching.py`.
- Se movió `candidato_compatible` y sus reglas relacionadas a un módulo dedicado.
- `main.py` ahora importa esa lógica en vez de contenerla directamente.
- Se agregaron tests unitarios de equivalencias.

Archivos:

- `app/matching.py`
- `app/main.py`
- `tests/test_matching.py`

### 3. Reducción segura de `main.py`

- Se retiró lógica de normalización y matching desde `main.py`.

- Los endpoints actuales se mantienen con las mismas rutas y estructura de respuesta.
- No se reestructuró la API ni se cambió la UX.

Archivo:

- `app/main.py`

## 4. Optimización de endpoints pesados

- `/diagnostico/calidad` ahora consulta columnas necesarias y usa `yield_per(500)` en vez de cargar objetos completos con `.all()`.
- `/estado-datos` ahora calcula conteos por supermercado con `GROUP BY` y conteos directos.
- Se mantuvo la forma de respuesta esperada por el frontend.

Archivo:

- `app/main.py`

## 5. Tests de integración mínimos

- Se agregó SQLite temporal en memoria para pruebas de API.
- Se prueba búsqueda básica con datos mínimos.
- Se prueba límite de resultados.
- Se prueba estado de datos con conteos básicos.

Archivo:

- `tests/test_integration.py`

## 6. Pydantic v2

- Se migró `@validator` a `@field_validator`.
- El warning de Pydantic v2 desaparece en la suite de tests.

Archivo:

- `app/schemas.py`

## 7. Dependencias

Se agregó `httpx`, requerido por `fastapi.testclient.TestClient` para los tests de integración.

Archivo:

- `requirements.txt`

## Comandos Ejecutados

```
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
```

## Resultados

- Tests: 11 passed.
- Compilación: OK.
- Import de FastAPI app: OK, salida FastAPI.

## Archivos Modificados o Creados

Modificados:

- `app/importar_csv.py`
- `app/main.py`
- `app/schemas.py`
- `requirements.txt`

Creados:

- `app/normalizacion.py`
- `app/matching.py`
- `tests/test_matching.py`
- `tests/test_integration.py`
- `CAMBIOS_FASE_2.md`

## Riesgos Pendientes

- Todavía quedan `.all()` en rutas livianas (categorias, subcategorias) y en lógica interna acotada; no se tocaron para mantener esta fase pequeña.
- `main.py` aún contiene lógica de búsqueda y resumen de compra que podría separarse en una fase posterior.
- El matching sigue siendo heurístico; requiere más fixtures reales para cubrir casos de marcas, sabores, formatos familiares y productos con nombres ambiguos.
- No se ejecutó pipeline completo de scrapers para evitar cambios masivos de datos.

## Siguiente Fase Recomendada

- 1 Extraer búsqueda de productos desde `main.py` a un service dedicado.
- 1 Agregar fixtures realistas para matching por supermercado.
- 1 Cubrir `/productos/resumen-compra` con tests de integración.
- 1 Revisar endpoints internos con `.all()` restantes y paginar si empiezan a crecer.
- 1 Evaluar carga explícita de `.env` local para mejorar experiencia del scraper sin exponer secretos.